Escola de Artes, Ciências e Humanidades
Bacharelado em Sistemas de Informação
ESI - Engenharia de Sistemas de Informação

Projeto da disciplina

API REST de uma máquina de café

1. Enunciado

Em grupos de 3 a 5 alunos, desenvolvam do zero o back-end de uma máquina de café automática, na forma de uma API REST. A máquina física tem um painel com botões e uma tela, mas esse painel é apenas um cliente: todas as regras e processamento da máquina (cardápio, estoque, pagamento, preparo e manutenção) estão na API, e é ela que vocês vão construir. Não é necessário criar interface gráfica nem hardware: a API será consumida por Postman, Insomnia, curl, Swagger ou via chamadas de uma classe controladora.

O objetivo não é fazer muitas funcionalidades, e sim entregar um software funcionando, testado e com código bem escrito, com evidências de qualidade e de que todos os integrantes do grupo participaram. Considerem a capacidade e o tempo do grupo, e selecionem as funções e as regras de negócio possíveis de serem implementadas, refatoradas e testadas no período disponível para a entrega do projeto. Todos os grupos implementam a mesma máquina, com liberdade para escolher linguagem, framework, banco de dados e arquitetura.

Alternativa sem HTTP: se não for possível fazer uma aplicação HTTP (por restrição de tecnologia, de ambiente ou de tempo), o grupo pode implementar a máquina como código local, com classes em Java, Python ou outra linguagem. Nesse caso, uma classe de entrada pública (por exemplo, MaquinaCafe) ocupa o lugar da API, e as demais exigências deste trabalho (regras de negócio, testes, qualidade, entregas e vídeo) continuam valendo.

2. Como a máquina funciona

Cardápio

A máquina prepara cinco bebidas. Cada uma consome ingredientes do estoque, tem preço e tempo de preparo (simulado).

Bebida	Água (ml)	Café (g)	Leite (ml)	Chocolate (g)	Preparo (s)	Preço
Café expresso	40	8	0	0	5	R$ 4,00
Café longo	120	8	0	0	8	R$ 4,50
Café com leite	40	8	150	0	10	R$ 6,00
Cappuccino	40	8	100	5	12	R$ 6,50
Chocolate quente	50	0	120	30	12	R$ 7,00

Além do ingrediente da receita, o cliente pode escolher a quantidade de açúcar: de 0 a 3 porções de 5 g cada, sem custo adicional. Cada bebida usa também 1 copo.

Estoque (reservatórios)

Item	Capacidade máxima	Alerta de nível baixo
Água	2000 ml	abaixo de 400 ml
Café em grãos	500 g	abaixo de 100 g
Leite	1000 ml	abaixo de 200 ml
Chocolate em pó	400 g	abaixo de 80 g
Açúcar	300 g	abaixo de 60 g
Copos	50 unidades	abaixo de 10 unidades

Ao iniciar pela primeira vez, todos os reservatórios estão cheios.

Estados da máquina

- **PRONTA:** aceita crédito e pedidos.
- **PREPARANDO:** há uma bebida em preparo; novos pedidos são recusados até o término.
- **AGUARDANDO_LIMPEZA:** atingido o limite de 30 preparos desde a última limpeza; pedidos bloqueados.
- **MANUTENCAO:** colocada pelo operador para reabastecer ou limpar; pedidos bloqueados.

Regras de negócio

- **RN01 – Crédito:** o cliente insere valores antes de pedir. Valores aceitos: R$ 0,50, 1,00, 2,00, 5,00, 10,00 e 20,00. O crédito acumulado não pode passar de R$ 50,00.
- **RN02 – Pedido:** só é aceito se a máquina estiver PRONTA, se o crédito cobrir o preço e se houver estoque suficiente de todos os ingredientes, do açúcar escolhido e de copo.
- **RN03 – Cobrança e consumo:** ao aceitar o pedido, o preço é debitado do crédito e os ingredientes são descontados do estoque. O que sobrar de crédito permanece disponível para um novo pedido.
- **RN04 – Preparo:** o pedido nasce com status EM_PREPARO e a máquina passa a PREPARANDO. Ao fim do tempo da receita, o pedido vira CONCLUIDO e a máquina volta a PRONTA. Sugestão: guardar o horário previsto de término e calcular o status na consulta, sem precisar de timer.
- **RN05 – Cancelamento:** só é permitido com o pedido EM_PREPARO. O valor é devolvido ao crédito, mas os ingredientes já consumidos não retornam ao estoque. A máquina volta a PRONTA.
- **RN06 – Troco:** o cliente pode pedir a devolução de todo o crédito restante a qualquer momento, o que zera o crédito.
- **RN07 – Limpeza obrigatória:** a cada 30 preparos concluídos desde a última limpeza, a máquina passa a AGUARDANDO_LIMPEZA. O operador deve entrar em manutenção, registrar a limpeza (o que zera o contador) e sair da manutenção.
- **RN08 – Manutenção:** o operador só entra em manutenção com a máquina PRONTA ou AGUARDANDO_LIMPEZA. Ao sair, a máquina volta a PRONTA, ou a AGUARDANDO_LIMPEZA se a limpeza ainda estiver pendente.

- **RN09 – Reabastecimento:** só em MANUTENCAO e nunca acima da capacidade máxima do reservatório.
- **RN10 – Cardápio e preços:** o operador pode alterar o preço de uma bebida (valor maior que zero). O cardápio informa se cada bebida está disponível, ou seja, se há estoque para prepará-la agora.
- **RN11 – Acesso do operador:** as operações de operador exigem autenticação por token. Um usuário e senha fixos, configurados por variável de ambiente, são suficientes.
- **RN12 – Relatório:** o operador consulta, por período, a quantidade vendida e o faturamento total e por bebida. Pedidos cancelados não entram.

Exemplo de fluxo

O cliente insere R$ 10,00 e pede um cappuccino (R$ 6,50) com 2 porções de açúcar. A API aceita o pedido, desconta 40 ml de água, 8 g de café, 100 ml de leite, 5 g de chocolate, 10 g de açúcar e 1 copo, e o crédito passa a R$ 3,50. Durante 12 segundos a máquina fica PREPARANDO e qualquer novo pedido recebe erro. Depois o pedido vira CONCLUIDO. O cliente pede o troco e recebe R$ 3,50 de volta, deixando o crédito em zero.

Endpoints mínimos (sugestão)

Os nomes e formatos podem variar, desde que estejam documentados e cubram as funcionalidades abaixo. Na versão em código local, cada endpoint corresponde a um método público da classe de entrada, os códigos HTTP correspondem a exceções (ou resultados de erro equivalentes) e a autenticação do operador pode ser feita por um parâmetro de credencial ou por um objeto de sessão.

Método	Rota	Descrição
GET	/produtos	Cardápio com preço, tempo de preparo e disponibilidade
GET	/status	Estado da máquina e crédito atual
POST	/credito	Insere valor de crédito
POST	/troco	Devolve todo o crédito restante
POST	/pedidos	Cria pedido (produto e porções de açúcar)
GET	/pedidos/{id}	Consulta pedido e seu status
POST	/pedidos/{id}/cancelar	Cancela pedido em preparo
POST	/operador/login	Autentica o operador e devolve o token
GET	/operador/estoque	Níveis de cada reservatório e alertas de nível baixo
POST	/operador/estoque/reabastecer	Repõe um ingrediente (apenas em manutenção)
PUT	/operador/produtos/{id}/preco	Altera o preço de uma bebida
POST	/operador/manutencao	Entra ou sai do modo de manutenção
POST	/operador/limpeza	Registra a limpeza (apenas em manutenção)
GET	/operador/relatorio	Vendas e faturamento por período

Códigos de erro esperados (exemplos)

Situação	Código HTTP sugerido
Bebida inexistente	404
Dados inválidos (açúcar fora de 0 a 3, valor de crédito não aceito)	400
Crédito insuficiente para o pedido	402
Estoque ou copo insuficiente; máquina preparando outra bebida	409
Máquina em manutenção ou aguardando limpeza	503
Operador sem token válido	401
Reabastecimento acima da capacidade	422

3. Requisitos de engenharia

- Código versionado em Git, em repositório compartilhado pelo grupo.
- Persistência em banco de dados (estoque, crédito, estado da máquina e pedidos devem sobreviver a uma reinicialização da API).
- Validação de entrada e tratamento padronizado de erros, conforme a tabela anterior.
- Documentação da API (Swagger/OpenAPI ou coleção Postman). Na versão local, documentação das classes e métodos públicos (Javadoc, docstrings ou equivalente).
- Testes automatizados (unitários e de integração) com cobertura mínima de 75% de código e de acordo com critérios de teste funcional ou caixa-preta
- Análise estática de código, com relatório de métricas.

Arquitetura e decisões de projeto

O grupo deverá separar, de forma coerente com a tecnologia utilizada, pelo menos as seguintes responsabilidades:

- **API/controllers:** receber requisições, tratar entradas e produzir respostas HTTP.
- **Serviços ou componentes de domínio:** implementar e coordenar as regras de negócio.
- **Persistência:** consultar e armazenar dados no banco de dados.

A separação não exige uma arquitetura complexa nem a criação de interfaces para todas as classes. O objetivo é evitar que regras de negócio fiquem concentradas nos controladores ou misturadas diretamente com detalhes de acesso ao banco.

Padrões de projeto

O grupo deverá identificar pelo menos uma situação em que um padrão de projeto possa ser útil. Deverá aplicar um padrão adequado ou justificar por que uma solução mais simples é suficiente. No documento de projeto, o grupo deve:

- Descrever o problema de projeto identificado.
- Indicar o padrão considerado e explicar como ele ajuda a resolver o problema, se aplicado.
- Explicar brevemente os benefícios e os custos ou complexidades introduzidos.

- Se optar por não aplicar o padrão, justificar por que a alternativa escolhida é adequada ao escopo.

Exemplos possíveis, conforme o domínio: Strategy para diferentes políticas de cálculo; State para transições de estado; Repository para separar o acesso a dados da lógica de negócio; Factory para criação de objetos quando houver variações relevantes. Os exemplos são sugestões, não uma lista de padrões que todos precisam implementar. A avaliação prioriza a adequação da decisão, e não a quantidade de padrões.

O grupo deverá incluir um diagrama simples de classes ou componentes, de no máximo uma página, coerente com a implementação entregue.

4. Entregas

#	Entrega	O que permite avaliar
1	Repositório Git com README (descrição, como rodar, como testar, decisões técnicas)	Produto, organização e reprodutibilidade
2	API em execução, com documentação Swagger ou coleção Postman que demonstre o fluxo completo da máquina (na versão local: código executável, com testes ou programa que demonstre o fluxo)	Funcionalidade e regras de negócio
3	Relatório de qualidade (1 a 2 páginas): cobertura de testes, resultado da análise estática (code smells, métricas de qualidade)	Código limpo e testes, com números
4	Documento de projeto: decisão sobre padrões de projeto (problema identificado, padrão considerado, benefícios e custos, ou justificativa para não aplicá-lo) e diagrama simples de classes ou componentes, de no máximo uma página, coerente com o código entregue	Arquitetura e adequação das decisões de projeto
5	Histórico de contribuições dos membros do grupo ao repositório	Participação de cada aluno
6	Declaração de contribuição individual: cada aluno descreve em poucas linhas o que fez (módulos, testes, revisões)	Divisão do trabalho, para cruzar com o histórico do Git
7	Vídeo (duração sugerida: 10 a 15 min) mostrando o código, a execução dos testes e as chamadas (requisições à API ou chamadas às classes. Para facilitar gravação e edição, pode ser gravado/narrado por apenas uma pessoa	Funcionamento comprovado e domínio do código

5. Exemplos de requisição e resposta

Os exemplos seguem o fluxo da seção 2, partindo de uma máquina recém-iniciada, e usam valores ilustrativos: nomes de campos, formatos e mensagens podem variar, desde que estejam documentados. Os identificadores de 1 a 5 seguem a ordem da tabela do cardápio (o cappuccino é o 4 e o chocolate quente é o 5).

Via API HTTP

1. Consultar o cardápio

```
GET /produtos

HTTP 200 OK
[
  {"id": 1, "nome": "Café expresso", "preco": 4.00, "disponivel": true},
  {"id": 4, "nome": "Cappuccino", "preco": 6.50, "disponivel": true},
  ...
]
```

2. Inserir crédito

```
POST /credito
{"valor": 10.00}

HTTP 200 OK
{"credito": 10.00}
```

3. Criar um pedido (cappuccino com 2 porções de açúcar)

```
POST /pedidos
{"produtoId": 4, "acucar": 2}

HTTP 201 Created
{
  "id": 1,
  "produto": "Cappuccino",
  "acucar": 2,
  "status": "EM_PREPARO",
  "preco": 6.50,
  "creditoRestante": 3.50,
  "previsaoTermino": "2026-03-10T14:30:12"
}
```

4. Novo pedido enquanto a máquina prepara (erro)

```
POST /pedidos
{"produtoId": 1, "acucar": 0}

HTTP 409 Conflict
{
  "erro": "MAQUINA_OCUPADA",
  "mensagem": "A máquina está preparando outra bebida."
}
```

5. Consultar o pedido após o tempo de preparo (12 segundos)

```
GET /pedidos/1

HTTP 200 OK
{"id": 1, "produto": "Cappuccino", "status": "CONCLUIDO"}
```

6. Pedido com crédito insuficiente (erro)

```
POST /pedidos
{"produtoId": 5, "acucar": 0}

HTTP 402 Payment Required
{
  "erro": "CREDITO_INSUFICIENTE",
  "mensagem": "Crédito de R$ 3,50 não cobre o preço de R$ 7,00."
}
```

7. Pedir o troco

```
POST /troco

HTTP 200 OK
{"valorDevolvido": 3.50, "credito": 0.00}
```

8. Consultar o estoque como operador, sem token (erro)

```
GET /operador/estoque

HTTP 401 Unauthorized
{"erro": "NAO_AUTORIZADO", "mensagem": "Token ausente ou inválido."}
```

9. Consultar o estoque como operador, com token

```
GET /operador/estoque
Authorization: Bearer <token>

HTTP 200 OK
{
  "reservatorios": [
    {"item": "agua", "atual": 1960, "capacidade": 2000, "nivelBaixo": false},
    {"item": "cafe", "atual": 492, "capacidade": 500, "nivelBaixo": false},
    {"item": "leite", "atual": 900, "capacidade": 1000, "nivelBaixo": false},
    {"item": "chocolate", "atual": 395, "capacidade": 400, "nivelBaixo": false},
    {"item": "acucar", "atual": 290, "capacidade": 300, "nivelBaixo": false},
    {"item": "copos", "atual": 49, "capacidade": 50, "nivelBaixo": false}
  ]
}
```

Via código local (classes)

O mesmo fluxo em uma versão sem HTTP. Os nomes de classes, métodos e exceções são ilustrativos.

6. Prazos e cronograma

Alguns pontos de controle:

- Conversa com o docente da disciplina sobre o andamento do projeto: 18/11
- Entrega do projeto final: 09/12

Cronograma sugerido para os grupos se organizarem:

- Definição dos grupos: 12 a 16/10
- Criação de infraestrutura técnica: 19 a 25/10
- Cardápio, estoque e crédito (CRUD): 26/10 a 01/11
- Operações mais complexas/transações: 02 a 29/11
- Fechamento, relatórios, gravação de vídeo: 03/11 a 08/12
- Entrega da documentação e vídeo: 09/12